

Agent Design Document

Personal Productivity and Task Execution Agent

Technical Design Document — Assignment 2

1. Problem Statement

Knowledge workers routinely juggle tasks, meeting notes, and planning across scattered tools — chat threads, notebooks, and calendars — with no single place that both stores this information and acts on it. Turning a meeting transcript into tracked action items, checking what is overdue, and building a realistic daily schedule are all manual, repetitive steps that consume time better spent on the work itself.

This project builds a tool-using AI agent that sits between the user and their task/notes data. Rather than behaving as a general-purpose chatbot, the agent interprets each request, decides whether stored data must be read or changed, executes the correct action through a well-defined tool, and asks for explicit approval before anything irreversible happens. The goal is a dependable assistant for day-to-day task and knowledge management — not a conversational novelty.

2. Users

The system is designed for a single individual managing their own workload — an “individual productivity” use case rather than a team or multi-tenant tool. Specifically:

- Individual professionals or students who want a lightweight, local task and notes system with an AI layer on top.
- Users who regularly sit in meetings and need action items extracted and turned into tracked tasks without manual re-typing.
- Users who want a daily or weekly planning assistant that reasons over their actual pending work rather than a generic template.

The current implementation assumes one user per running instance (single local SQLite database, no authentication/multi-tenant layer), which matches the assignment’s scope of simulated/local data.

3. Use Cases

- Create a task directly (“Create a task: buy groceries, high priority, due tomorrow”).
- List or filter tasks by status, priority, due date, or tag (“Show me all high-priority tasks due this week”).
- Update or complete an existing task, with approval before the change is applied.
- Save and later search notes by keyword, category, or date range.
- Extract a structured summary — decisions, action items, owners, deadlines, open questions — from pasted meeting notes.
- Turn extracted meeting actions into approved tasks in one workflow.
- Generate an ordered daily work plan from current pending tasks and available hours.
- Produce a weekly report covering completed, overdue, and blocked tasks with next-week recommendations.
- Detect overdue tasks and get a recommendation on what to work on first.
- Create a reminder, or draft (never auto-send) a follow-up email based on meeting notes.
- Ask a direct conceptual question (e.g. “What’s the difference between High and Critical priority?”) and get an answer with no tool call and no data side-effect.

- Resolve a follow-up reference such as “mark the second one as complete” against the previous turn’s tool result.

4. Agent Responsibilities

The agent is allowed to:

- Interpret a natural-language request and decide, per turn, whether a tool is required at all.
- Select one or more of the twelve registered tools and supply validated arguments to them.
- Read task and note data (list, search, detect overdue) without requiring approval, since these are non-destructive.
- Create a single new task or note, save a reminder proposal, draft (but not send) an email, or generate a plan/report — all scoped to the current user’s own local data store.
- Chain multiple tool calls within one turn, up to the configured step limit, to satisfy multi-step requests (e.g. meeting notes → proposed tasks → created tasks).
- Ask a clarifying question when a request is ambiguous in a way that changes which data or tool would be used.
- Log every step of its own execution (tool, arguments, result, approval outcome, timing) for later review.

5. Agent Boundaries

The agent is explicitly not allowed to:

- Execute any write action — update, complete, create a reminder, or create multiple tasks at once — without the user’s explicit approval of the exact proposed action first.
- Actually send an email, notification, or any external communication; “draft follow-up email” only ever produces text for the user to review and send themselves.
- Delete any record. No delete tool exists in the current tool catalogue.
- Call a tool when a direct, ungrounded answer fully satisfies the request (e.g. definitional questions).
- Invent task IDs, note IDs, counts, dates, or note content that did not come from an actual tool result or the user’s own message.
- Exceed the configured execution limits — maximum agent steps, per-tool retries, or repeat an identical tool call — regardless of how the model would otherwise behave.
- Expose its internal chain-of-thought, tool schemas, system instructions, stack traces, or the configured API key through the user interface.
- Access data belonging to any user other than the one running the local instance; there is no cross-session or cross-user data sharing.

6. Tool Catalogue

Twelve tools are implemented: the eight required tools plus four bonus tools (create reminder, draft follow-up email, detect overdue tasks, generate weekly report). Every tool has a Pydantic input schema, so malformed arguments are rejected before execution rather than causing a runtime error inside the tool.

Tool	Purpose	Key Inputs	Output	Approval
create_task	Create a new task record	title, description, priority, due_date, tags	created task, task_id, confirmation	Not required (single task)
list_tasks	Query tasks with filters	status, priority, due_date, tag	matching tasks, total count	Not required (read-only)

Tool	Purpose	Key Inputs	Output	Approval
update_task	Modify an existing task's fields	task_id, changed fields	updated task	Required
complete_task	Mark a task completed	task_id	updated status, completion timestamp	Required
search_notes	Keyword search over saved notes	query, category, date range	ranked matching notes	Not required (read-only)
save_note	Persist a new note	title, content, category, tags	note_id, confirmation	Not required
extract_meeting_actions	Turn a transcript into a structured summary	meeting notes / transcript text	summary, decisions, action items, owners, deadlines, open questions	Not required (proposal only; creating tasks from it is)
generate_work_plan	Build an ordered daily schedule	available_hours, date, task list, priorities	ordered schedule, focus areas, deferred tasks, risk warnings	Not required (read-only)
create_reminder	Create a reminder for a task or date	title, remind_at, related task_id	reminder_id, confirmation	Required
draft_follow_up_email	Draft (not send) a follow-up email	meeting notes / context, recipient	subject, body draft	Not required (drafts only, nothing is sent)
detect_overdue_tasks	Find tasks past their due date	optional status/priority filter	overdue tasks, days overdue	Not required (read-only)
generate_weekly_report	Summarize the week's task activity	date range	completed/overdue counts, blocked items, next-week priorities	Not required (read-only)

7. State Model

The application keeps two categories of state: data the user owns (tasks and notes, persisted to disk) and execution state scoped to a run or session (kept in memory only). No sensitive configuration — such as the API key — is ever included in any of these records.

State	Contents	Lifetime / Storage
Task records	ID, title, description, priority, status, due/created/updated dates, tags, source, notes	Persistent — SQLite database file
Note records	ID, title, content, category, tags, created/updated dates	Persistent — SQLite database file
Conversation / session memory	Recent messages, tool outputs, referenced task/note IDs, stated preferences	In-memory for the active session; cleared on “Clear conversation” or app restart
Pending approval	Proposed tool name, arguments, expected effect	In-memory; resolved (approve/reject/edit) before the agent continues
Execution log entry	Run ID, request, model, tools called, arguments, results, approval status, errors, start/end time, duration, outcome	Persistent — written per run for later review; never includes API keys or raw reasoning
Agent run counters	Step count, per-tool retry count, recent tool-call signatures	In-memory, scoped to a single agent run; used for step/retry/duplicate limits

8. Approval Model

Approval is required before any action that changes stored data in a way the user did not directly and specifically type themselves, or that would be difficult to reverse. Concretely, the following require explicit human approval before the tool actually executes:

- Updating an existing task (`update_task`).
- Completing a task (`complete_task`).
- Creating a reminder (`create_reminder`).
- Creating multiple tasks in one action, such as converting extracted meeting actions into tasks.
- Any other action classified as irreversible or bulk by nature, even if not listed above by name.

Read-only tools (`list_tasks`, `search_notes`, `detect_overdue_tasks`, `generate_work_plan`, `generate_weekly_report`) and single-item creations that the user asked for explicitly and unambiguously (`create_task`, `save_note`, `draft_follow_up_email` — which only drafts text) do not require approval, since they are either non-destructive or exactly match a specific, explicit user instruction with no hidden side-effect.

When approval is required, the interface shows the proposed action, tool name, exact input arguments, and expected effect, with Approve, Reject, and — where practical — Edit options. Rejecting an action returns control to the user immediately; the tool is never executed, and the agent does not silently retry the same action.

9. Error Strategy

Every category of failure below is caught at a specific layer of the system, converted into a short, non-technical message, and logged with enough internal detail (exception type, tool name, retry count) for debugging — without ever surfacing a raw stack trace or secret to the user.

Failure	Handling	User-facing message
Missing API key	Detected at startup / first call; raises a dedicated <code>MissingAPIKeyError</code> before any request is sent	“Gemini API key is not configured — add it to your <code>.env</code> file.”
Invalid model response / empty response	Response validated for text or function calls; if neither is present, the run stops with a diagnostic (finish reason) logged internally	“The AI service returned an empty response.”
Invalid tool arguments	Pydantic schema validates every tool call before execution; invalid calls never reach the tool	“That request was missing required information: ...”
Unknown task ID	Lookup fails inside the tool; tool returns a structured error instead of raising	“I couldn’t find a task with that ID.”
Database failure	Caught at the storage layer; run marked failed, no partial writes committed	“Something went wrong saving your data — please try again.”
Empty user input	Rejected before reaching the agent loop	“Please enter a request.”
Tool execution timeout	Per-tool timeout (30s default) cancels the call	“That action took too long and was stopped.”
LLM API error (rate limit / 429)	Retried automatically with backoff (using the server’s suggested delay) up to 3 times before surfacing an error	“Gemini quota exceeded — please check your plan or try again shortly.”
Unsupported request	Agent answers directly that it cannot perform the action rather than forcing a tool call	Plain-language explanation of what the agent can and cannot do
Repeated failed tool calls	Retry counter per tool (max 2); duplicate identical calls are detected and blocked	“This action failed repeatedly and was stopped to avoid a loop.”

Execution limits (documented in docs/LIMITS.md) work alongside this error strategy to bound worst-case behavior: a maximum of 8 agent steps per turn, up to 2 retries per tool, a 30-second timeout per tool call, duplicate tool-call detection, and an overall loop-prevention check on the agent controller. These defaults balance letting genuinely multi-step requests (like the meeting-notes-to-tasks workflow) complete, against stopping a misbehaving loop before it burns excessive API quota or time.

10. Security Considerations

Secrets: The Gemini API key is read only from an environment variable (via a local .env file, excluded from version control) and is never written into the database, the execution log, or any UI element. Error messages are sanitized before display so an underlying exception cannot leak the key.

Permissions: The agent operates with exactly the permissions its tool definitions grant it — there is no direct database or filesystem access outside the tool layer, and no tool exists to delete data or reach outside the local task/notes store. Every tool call is validated against its Pydantic schema before it can touch storage.

User data: Tasks and notes are stored locally in a single SQLite file scoped to one user; there is no network transmission of this data except the minimum necessary content sent to the Gemini API to generate a response. No task or note content is logged in plaintext anywhere beyond the local execution log, which itself is for the user’s own review.

Approval as a security control: Requiring explicit human approval before any write, update, or simulated-send action is itself a security boundary — it prevents a misinterpreted instruction or a model error from silently mutating or exfiltrating data.

Execution limits as abuse prevention: Step, retry, and duplicate-call limits also protect against runaway loops that could otherwise be used (accidentally or adversarially) to exhaust API quota or hammer the local database.

Least exposure in the UI: The interface deliberately shows only short operational status (“Thinking”, “Waiting for approval”, etc.) and never the model’s internal chain-of-thought or the raw tool-calling protocol, reducing the information available to a user attempting to manipulate the agent’s behavior.